

Einlesen & Überblick

```
import pandas as pd
```

```
df = pd.read_csv("../data/raumlufte.csv",
delimiter=";")
df.head() # erste 5 Zeilen
df.dtypes # Datentyp pro Spalte
df.describe() # count, mean, std, min,
max ...
```

- Alles in **einer** Spalte? → Trennzeichen fehlt: `delimiter=";"` oder `"\t"`
- Pfad relativ zum Notebook: `../data/...` (Ordner data neben dem Blockordner)

Spalten, Filter, Aggregation

```
df["co2"] # eine Spalte → Series
df[["co2", "temperature"]] # Liste →
DataFrame
df.filter(like="temp") # Spalten mit
"temp" im Namen
df.rename(columns={"temperature":
"temp_c"})

df["temp_f"] = df["temperature"] * 9 / 5
+ 32 # neues Feature

df[df["co2"] > 1000] # Zeilen filtern
df[(df["co2"] > 1000) & (df["pir"] == 1)]
# und: &, oder: |
df[df["alarm"].isin([1, 2])]

df["co2"].agg(["mean", "min", "max"])
```

- Bedingungen mit `&` / `|` immer in Klammern setzen

Fehlende Werte, Duplikate, Ausreisser

```
df.isna().sum() # Lücken pro Spalte
df.duplicated(subset=["timestamp"]).sum()
df = df.drop_duplicates(subset=
["timestamp"], keep="first")

df["alarm"] = df["alarm"].fillna(0) #
fehlt = kein Alarm
df["co2"] =
df["co2"].interpolate(method="linear") #
kurze Lücke
df = df.dropna(subset=["temperature"])

z = (df["temperature"] -
df["temperature"].mean()) /
df["temperature"].std()
df = df[z.abs() <= 3] # z-Score:
Ausreisser weg
```

- `fillna(0)` nur, wenn 0 fachlich stimmt (kein Eingriff, kein Alarm)
- Echte Sensorlücken sichtbar lassen oder begründet füllen

Datentypen & Zeitindex

```
df["light"] = df["light"].astype(int)
df["timestamp"] =
pd.to_datetime(df["timestamp"],
format="%d.%m.%Y %H:%M")
df =
df.set_index("timestamp").sort_index()

df.loc["2026-01-15"] # ein Tag
df.loc["2026-01-15 08:00":"2026-01-15
12:00"]
df.index.hour # Stunde je Zeile
df.index[0] + pd.Timedelta(hours=1)

df_utc =
df.tz_localize("Europe/Zurich").tz_conver
t("UTC")
```

- Zeitstempel sind nach dem Einlesen oft Text (object/str) → `to_datetime`
- `%d.%m.%Y %H:%M` = 15.01.2026 07:30 · `%Y-%m-%d` = 2026-01-15
- Sommerzeit: 02:00–02:59 fehlt im März, 02:00–02:59 doppelt im Oktober

Resampling braucht DatetimeIndex

```
df_1h = df[["co2",
"temperature"]].resample("1h").mean() #
Down
df_5min =
df_1h.resample("5min").interpolate() #
Up
```

Aggregation wann

<code>mean()</code>	Zustandsgrößen: Temperatur, CO2, Leistung
<code>sum()</code>	Mengen: Energie [kWh], Anzahl Ereignisse
<code>max()</code> / <code>min()</code>	Spitzen, Grenzwertverletzungen, Frost

- Kürzel: "15min" "1h" "1D" "1W" "1ME" (Monat) · nicht "1H"

Joins & Imputation

```
co2 = pd.read_csv("../data/co2.csv",
index_col="timestamp", parse_dates=True)
temp =
pd.read_csv("../data/temperature.csv",
index_col="timestamp", parse_dates=True)
co2 = co2.rename(columns={"value":
"co2"})
temp = temp.rename(columns={"value":
"temperature"})

both = co2.join(temp, how="outer") #
verbindet über den Index
both["temperature"] =
both["temperature"].interpolate(method="time")
```

how= welche Zeitpunkte bleiben?

inner	nur die in beiden Tabellen
left / right	alle der linken / rechten Tabelle
outer	alle aus beiden, Lücken werden NaN

- Gleiche Spaltennamen → vorher `rename` (sonst „columns overlap“)
- **Interpolieren** passt bei stetigen Messgrößen und kurzen Lücken (Temperatur, CO2)
- **Fragwürdig** bei Zuständen und Zählern (Präsenz pir, Alarm, Schalter) und langen Lücken → `fillna`, `ffill()` oder Lücke lassen

Pivot / Melt

```
wide = df[["co2",
"temperature"]].reset_index()
long = pd.melt(wide, id_vars=
["timestamp"], var_name="sensor",
value_name="wert")
wide = long.pivot(index="timestamp",
columns="sensor", values="wert")
```

- **melt**: breit → lang (eine Zeile pro Messwert) · **pivot**: lang → breit

Univariate Statistik

```
df["temperature"].describe()
df["temperature"].mean() # Mittelwert
df["temperature"].median() # robust
gegen Ausreisser
df["temperature"].std() # Streuung
df.groupby(df.index.hour)["co2"].mean()
# Mittel je Stunde
```

- Höherer **mean** → im Mittel grösser · höhere **std** → schwankt stärker
- mean ≠ median deutlich → schiefe Verteilung oder Ausreisser

Plotly

```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(x=df.index,
y=df["co2"], mode="lines", name="CO2"))
fig.update_layout(title="CO2 im Raum",
xaxis_title="Zeit", yaxis_title="CO2
[ppm]")
fig.show()

go.Histogram(x=df["co2"], nbinsx=20)
go.Bar(x=["A", "B"], y=[23.8, 22.9])
go.Scatter(x=df["light"], y=df["co2"],
mode="markers")
```

Frage	Plot
Verlauf über die Zeit	Linie Scatter(mode="lines")
Verteilung, Ausreisser	Histogram
Kategorien vergleichen	Bar
Zusammenhang zweier Grössen	Punktwolke mode="markers"

- Mehrere Panels: `make_subplots(rows=2, cols=1, shared_xaxes=True)` · Warnbereich: `fig.add_hrect(y0=800, y1=1000)`
- Fehler „requires nbformat“ → %pip install nbformat, Kernel neu starten

Korrelation

```
df[["co2", "temperature",
"light"]].corr() # Pearson
df[["co2", "temperature",
"light"]].corr(method="spearman")
```

	misst	passt bei
Pearson	linearen Zusammenhang	Punktwolke ≈ Gerade
Spearman	monotonen Zusammenhang (Ränge)	steigt stetig, aber gekrümmt; Ausreisser

- Wert von -1 bis +1: nahe ±1 stark, nahe 0 kein (linearer/monotoner) Zusammenhang
- Spearman deutlich > Pearson → monoton, aber nicht linear
- Korrelation ist **keine** Kausalität

Lineare Regression

```
from sklearn.linear_model import
LinearRegression
from sklearn.metrics import
mean_absolute_error, mean_squared_error
```

```
X = df[["light"]] # Merkmal(e): 2D,
doppelte Klammern
y = df["co2"] # Zielvariable
model = LinearRegression()
model.fit(X, y)
model.coef_, model.intercept_ # Steigung
m, Achsenabschnitt b
```

```
df["prediction"] = model.predict(X)
df["residuals"] = df["co2"] -
df["prediction"]
mean_absolute_error(df["co2"],
df["prediction"])
mean_squared_error(df["co2"],
df["prediction"])
```

- **Steigung** `coef_`: Änderung von y, wenn x um 1 Einheit steigt (positiv: mehr x → mehr y)
- **Residuum** = echter Wert – Vorhersage
- Residuen zufällig um 0 → Gerade passt · Bogen/Muster → Zusammenhang nicht linear · Trichter → Fehler wächst mit x
- **MAE**: mittlerer absoluter Fehler, in der Einheit von y
- **MSE**: Fehler quadriert → grosse Fehler zählen überproportional, Einheit²
- Was „gut“ ist, hängt vom Einsatz ab (reichen ±50 ppm? ±2 °C?)

Weitere Modelle & Train/Test

```
from sklearn.model_selection import
train_test_split
from sklearn.tree import
DecisionTreeRegressor
from sklearn.neighbors import
KNeighborsRegressor
```

```
train_df, test_df = train_test_split(df,
test_size=0.2, random_state=42)
```

```
model =
DecisionTreeRegressor(max_depth=4) #
oder KNeighborsRegressor(n_neighbors=5)
model.fit(train_df[["light"]],
train_df[["co2"]])
train_mse =
mean_squared_error(train_df["co2"],
model.predict(train_df[["light"]]))
test_mse =
mean_squared_error(test_df["co2"],
model.predict(test_df[["light"]]))
```

Befund	Bedeutung
Train-MSE klein, Test-MSE viel grösser	Overfitting: auswendig gelernt
Train- und Test-MSE beide hoch	Underfitting: Modell zu einfach
beide tief und ähnlich	gute Generalisierung

- Modell nach dem **Test**-MSE wählen, nicht nach dem Train-MSE
- Testset nie zum Trainieren benutzen: es simuliert neue, unbekannte Daten
- Overfitting-Gefahr: grosses `max_depth`, kleines `n_neighbors` (1 = auswendig)

Typische Fehlermeldungen

FileNotFoundError	Pfad/Ordner prüfen: ../data/...
KeyError: 'xy'	Spaltenname falsch → df.columns
„Only valid with DatetimeIndex“	vor resample: to_datetime + set_index
„Expected a 2- dimensional container“	X = df[["light"]] statt df["light"]
„Input X contains NaN“	Lücken vor fit füllen oder entfernen
„columns overlap“	beim join gleiche Spaltennamen → rename
ModuleNotFoundError	%pip install ... in Zelle, Kernel neu starten